

Auditoría Completa — Backend-Travel-Agency-2

Fecha: 16 de mayo de 2026
Proyecto: Backend-Travel-Agency-2 (Spring Boot + Maven + MySQL)
Stack real: Java 25 + Spring Boot 4.0.6 + Maven + MySQL

1. Resumen Ejecutivo

El proyecto es una API RESTful para una agencia de viajes. Está bien estructurado en capas (controller → service → repository → entity) con manejo de excepciones centralizado, DTOs, mappers, y un motor de precios robusto con descuentos escalables.

El proyecto pasó de **6** ● **críticos** (auditoría del 15/05) a **3** ● **críticos** actualmente. Se corrigieron los problemas graves de seguridad JWT y contraseñas en texto plano, pero persisten issues de calidad media/alta y faltan funcionalidades clave del briefing.

Puntuación general: 7/10

2. Stack Tecnológico

Tecnología	Versión
Java	25
Spring Boot	4.0.6 (via parent)
Spring Data JPA	Hibernate
MySQL	8+ (mysql-connector-j)
Maven	3.9.14 (wrapper)
Lombok	Última (via Spring Boot)
Cloudinary	1.39.0
Auth0 java-jwt	4.4.0
jBCrypt	0.4
SpringDoc OpenAPI	2.8.5
Tests	JUnit 5 + Mockito

 El README.md indica **Java 17 + Spring Boot 3.2.5**, que está **desactualizado**.

3. Estructura del Proyecto

```
src/main/java/com/inditex/g1_agencia_viajes/  
├─ G1AgenciaViajesApplication.java      # Entry point  
├─ config/                             # Configuraciones  
│   └─ CloudinaryConfig.java
```

```
|   └─ CorsConfig.java
└─ controller/                                # 11 controladores
    │   └─ AuthenticationController.java
    │   └─ BookingController.java
    │   └─ BusController.java
    │   └─ CloudinaryController.java
    │   └─ DriverController.java
    │   └─ EmployeeController.java
    │   └─ HotelController.java
    │   └─ OfferController.java
    │   └─ TravelController.java
    │   └─ TripSegmentController.java
    │   └─ UserController.java
└─ dto/                                        # 22 DTOs
    │   └─ BookingQuotePassengerDetailDTO.java
    │   └─ BookingQuoteRequestDTO.java
    │   └─ BookingQuoteResponseDTO.java
    │   └─ BookingRequestDTO.java
    │   └─ BookingResponseDTO.java
    │   └─ BookingUserRequestDTO.java
    │   └─ BusRequestDTO.java
    │   └─ BusResponseDTO.java
    │   └─ DriverRequestDTO.java
    │   └─ DriverResponseDTO.java
    │   └─ HotelRequestDTO.java
    │   └─ HotelResponseDTO.java
    │   └─ LoginRequest.java
    │   └─ LoginResponse.java
    │   └─ OfferRequestDTO.java
    │   └─ OfferResponseDTO.java
    │   └─ TravelRequestDTO.java
    │   └─ TravelResponseDTO.java
    │   └─ TripSegmentRequestDTO.java
    │   └─ TripSegmentResponseDTO.java
    │   └─ UserRequestDTO.java
    │   └─ UserResponseDTO.java
└─ exception/                                # 9 excepciones + handler
    │   └─ BusFullException.java
    │   └─ DriverOverlapException.java
    │   └─ DuplicateLicensePlateException.java
    │   └─ EmailAlreadyExistsException.java
    │   └─ GlobalExceptionHandler.java
    │   └─ HotelNotAvailableException.java
    │   └─ MinorWithoutTutorException.java
    │   └─ PastTravelException.java
    │   └─ ResourceNotFoundException.java
    │   └─ TravelNotAvailableException.java
└─ mapper/                                    # 8 mappers
    │   └─ BookingMapper.java
    │   └─ BookingUserMapper.java
    │   └─ DriverMapper.java
    │   └─ HotelMapper.java
```

```

|   |─ OfferMapper.java
|   |─ TravelMapper.java
|   |─ TripSegmentMapper.java
|   └─ UserMapper.java
└─ model/                                # 9 entidades + 3 enums
    |─ Booking.java
    |─ Bus.java
    |─ Driver.java
    |─ Employee.java
    |─ Gender.java (enum)
    |─ Hotel.java
    |─ Offer.java
    |─ Role.java (enum)
    |─ Travel.java
    |─ TripSegment.java
    |─ TypeBoard.java (enum)
    └─ User.java
└─ repository/                          # 9 repositorios JPA
    |─ BookingRepository.java
    |─ BusRepository.java
    |─ DriverRepository.java
    |─ EmployeeRepository.java
    |─ HotelRepository.java
    |─ OfferRepository.java
    |─ TravelRepository.java
    |─ TripSegmentRepository.java
    └─ UserRepository.java
└─ security/                            # Seguridad JWT
    |─ JwtFilter.java
    |─ JwtUtil.java
    └─ SecurityConfig.java
└─ service/                             # 12 servicios
    |─ BookingPricingService.java
    |─ BookingService.java
    |─ BusService.java (interface)
    |─ BusServiceImpl.java
    |─ CloudinaryService.java
    |─ DriverService.java
    |─ EmployeeService.java
    |─ HotelService.java
    |─ OfferService.java
    |─ TravelService.java
    |─ TripSegmentService.java
    └─ UserService.java

```

4. Modelo de Datos (BD)

Tabla	Propósito	FK
users	Cientes del sistema	tutor_id → users

employees	Empleados/agentes	—
hotels	Hoteles disponibles	—
buses	Autobuses	—
drivers	Conductores	—
offers	Ofertas temporales	—
travels	Viajes	hotel_id → hotels, offer_id → offers
trip_segments	Trayectos del viaje	travel_id → travels, bus_id → buses, driver_id → drivers
bookings	Reservas	travel_id → travels, employee_id → employees
customers_bookings	N:N bookings ↔ users	booking_id, customer_id

5. Escala de Valoración

- **Crítico:** Impide el funcionamiento o es un riesgo de seguridad grave. Arreglar inmediatamente.
- **Alto:** Impacta negativamente en calidad, mantenibilidad o funcionalidad. Arreglar pronto.
- **Medio:** Incumple buenas prácticas o convenios del proyecto. Mejorar cuando se pueda.
- **Punto fuerte:** Aspecto positivo del proyecto.

6. Issues Críticos ●

C1 — JWT secret con fallback hardcodedo

Archivo: application.properties:23

```
jwt.secret=${JWT_SECRET:default_secret_change_me}
```

Si la variable de entorno `JWT_SECRET` no está definida, se usa `default_secret_change_me`. Cualquiera puede forjar tokens JWT válidos.

Solución: Quitar el `:default_secret_change_me` para que falle si no está definida la env var.

C2 — EmployeeController expone entidad sin DTO

Archivo: EmployeeController.java:36,41

```
public ResponseEntity<Employee> create(@RequestBody Employee employee) { ... }
public ResponseEntity<Employee> update(@PathVariable Long id, @RequestBody Employee
employee) { ... }
```

Acepta y devuelve `Employee` directamente:

- Bypassa validaciones (`@Valid` no está presente)
- Expone el modelo interno (incluyendo `password` hasheada)
- No separa representación interna de externa

Solución: Crear `EmployeeRequestDTO` y `EmployeeResponseDTO` y usarlos en estos métodos.

C3 — Sin catch-all Exception handler

Archivo: `GlobalExceptionHandler.java`

Actualmente maneja estas excepciones:

- `MethodArgumentNotValidException` → 400
- `ResourceNotFoundException` → 404
- `HotelNotAvailableException`, `TravelNotAvailableException`, `EmailAlreadyExistsException`, `DuplicateLicensePlateException`, `DriverOverlapException`, `PastTravelException`, `BusFullException` → 409
- `IllegalArgumentException`, `MinorWithoutTutorException` → 400

No maneja:

- `Exception` (catch-all) — stack trace 500 al cliente
- `HttpMessageNotReadableException` — JSON malformado
- `MethodArgumentTypeMismatchException` — tipo de variable incorrecto
- `ConstraintViolationException` — validaciones JPA
- `DataIntegrityViolationException` — violaciones DB
- `MissingServletRequestPartException` — subida de archivos
- `AccessDeniedException` — seguridad

Solución: Añadir handler genérico `@ExceptionHandler(Exception.class)` que devuelva 500 sin trazas internas.

7. Issues Altos

H1 — `TravelService` usa `findAll()` + filtro en memoria

Archivo: `TravelService.java:35-66`

`getAll()`, `getAvailable()` y `getOnSale()` usan `travelRepository.findAll()` y filtran con `stream()`. El repositorio tiene 3 queries derivadas:

- `findByActiveTrue()`
- `findByActiveTrueAndStartDateAfter()`
- `findBySaleTrueAndActiveTrueAndStartDateAfter()`

Ninguna se usa. La versión inicial del proyecto (commit `929a41d`) sí las usaba.

H2 — `Collections.shuffle()` en listados

Archivo: `TravelService.java:37,50,62`

Añadido en commit `7651cef`. Baraja aleatoriamente los resultados:

- Hace impredecible el orden de respuesta
 - Inhabilita paginación
 - Es efecto secundario sobre la lista original
-

H3 — `getAll()` sin filtrar activos

Archivo: `TravelService.java:35-41`

Regresión desde commit `69fc944` . Antes usaba `findByActiveTrue()` . Ahora devuelve viajes inactivos y pasados.

H4 — `getOnSale()` sin filtrar por fecha

Archivo: `TravelService.java:57-66`

Regresión desde commit `69fc944` . Antes usaba `findBySaleTrueAndActiveTrueAndStartDateAfter()` . Ahora muestra ofertas de viajes ya pasados.

H5 — 5 repositorios sin `@Repository`

Archivos: `BookingRepository.java` , `DriverRepository.java` , `EmployeeRepository.java` , `OfferRepository.java` , `TripSegmentRepository.java`

Funcionan por herencia de `SimpleJpaRepository` pero la anotación `@Repository` es una buena práctica que además permite traducción de excepciones JPA.

H6 — `EmployeeService` sin `@Transactional`

Archivo: `EmployeeService.java`

Todos los métodos carecen de `@Transactional` . Las operaciones de escritura (save, update, delete) no están envueltas en una transacción.

H7 — Falta `@Max(5)` en `stars`

Archivo: `HotelRequestDTO.java` , `Hotel.java`

```
@Min(value = 1)
private int stars;
```

Sin `@Max(5)` se puede crear un hotel de 100 estrellas.

H8 — Falta `@NotEmpty` en `customerIds`

Archivo: `BookingRequestDTO.java:25`

```
private List<Long> customerIds;
```

Sin validación de tamaño mínimo, se podría crear una reserva sin clientes.

H9 — Faltan `@Pattern` en validaciones de formato

Archivo	Campo	Problema
<code>BusRequestDTO.java</code>	<code>licensePlate</code>	Sin patrón (ej. <code>^[0-9]{4}[A-Z]{3}\$</code>)
<code>DriverRequestDTO.java</code>	<code>phone</code>	Sin <code>@Pattern</code> para teléfono

UserRequestDTO.java	passport	Sin validación de formato
---------------------	----------	---------------------------

H10 — BookingService.update() no reconcilia capacidad

Archivo: BookingService.java:88-114

Al cambiar de viaje o modificar clientes en una reserva existente, no restaura las plazas del hotel/viaje anterior ni valida la capacidad del nuevo.

H11 — Swagger solo en CloudinaryController

10 de 11 controllers carecen de anotaciones `@Operation`, `@ApiResponses` y `@Schema`. Solo `CloudinaryController.java` tiene documentación Swagger completa.

H12 — README desactualizado

Archivo: README.md

El README referencia:

- Java 17 (→ real: Java 25)
- Spring Boot 3.2.5 (→ real: 4.0.6)
- Faltan controllers: Offer, TripSegment, Cloudinary, Authentication
- Estructura del proyecto incompleta
- Tablas de entidades con campos desactualizados

8. Issues Medios

M1 — Nombres en español

Archivo	Nombre incorrecto	Corrección
HotelService.java	reducirPlazas()	reduceCapacity()
HotelService.java	liberarPlazas()	releaseCapacity()
JwtUtil.java	campo algoritmo	algorithm
JwtUtil.java	método crearToken()	createToken()
JwtUtil.java	método getAlgoritmo()	getAlgorithm()

Violación de `AGENTS.md` sección 2 (nombres en inglés).

M2 — UserService usa hard delete

Archivo: UserService.java

Inconsistente con el resto de entidades:

Entidad	Tipo de borrado
Hotel, Bus, Driver, Travel	Soft delete (<code>active = false</code>)

User	Hard delete (<code>repository.deleteById()</code>)
Offer, TripSegment	Hard delete

M3 — BusServiceImpl sin mapper dedicado

Archivo: `BusServiceImpl.java`

Único servicio que no usa una clase Mapper separada. La conversión entre entidad y DTO se hace inline en métodos privados `toResponseDTO()` y `toEntity()`.

M4 — UserResponseDTO redundante

Archivo: `UserResponseDTO.java`

- `@Data` ya incluye `@Getter`, `@Setter`, `@ToString`, `@EqualsAndHashCode`. Las anotaciones extra son redundantes.
- Campos duplicados: `userId` y `id` parecen representar lo mismo.
- `fullName` nunca se popula.

M5 — N+1 queries en bucle

Archivo	Método	Problema
<code>BookingService.java:233</code>	<code>resolveCustomersByIds()</code>	<code>findById()</code> en loop
<code>BookingPricingService.java:66</code>	<code>loadUsers()</code>	<code>findById()</code> en loop

Ambos deberían usar `findAllById()` para una sola consulta.

M6 — Typos en mensajes de excepción

En múltiples servicios: "I bus", "I hotel", "I cliente", "I viaje", "I empleado", "I tutor", "I trayecto" — falta la letra "e" de "el".

M7 — Endpoint naming mixto

- Español: `/activos`, `/disponibles`
- Inglés: `/available`, `/sale`, `/quote`
- Sin versionado de API (`/api/v1/`)

M8 — TripSegmentServiceTest — mock faltante

Archivo: `TripSegmentServiceTest.java`

`tripSegmentRepository.findOverlappingByDriver()` no está mockeado en `create()` / `update()`. La lógica de solapamiento de conductores NO se testea.

M9 — G1AgenciaViajesApplicationTests sin base de datos de test

Archivo: `G1AgenciaViajesApplicationTests.java`

@SpringBootTest sin H2 ni perfil test. Intenta conectar a MySQL real para cargar el contexto.

M10 — TravelServiceTest usa RuntimeException genérica

Archivo: TravelServiceTest.java:147

Usa RuntimeException.class en lugar de ResourceNotFoundException.class .

9. Puntos Fuertes

Aspecto	Detalle
✓ Seguridad JWT corregida	Filter autentica, rutas públicas limitadas a /api/authentication/login, /api-docs, /swagger-ui
✓ Secret JWT en properties	@Value("\${jwt.secret}") en JwtUtil y JwtFilter (ya no hardcodeado)
✓ Tokens con expiración	24h configurable via jwt.expiration
✓ Login por email	AuthenticationController usa LoginRequest.email + BCrypt
✓ Passwords BCrypt en seed	data.sql con hashes BCrypt válidos
✓ Constructor injection	Sin @Autowired en campos en todo el código
✓ Motor de precios robusto	BookingPricingService con child/pensioner/group/offer discounts + endpoint /api/bookings/quote
✓ Cloudinary funcional	Upload + delete con config externalizada
✓ CORS configurado	localhost:5173-5177
✓ DTOs separados	22 DTOs para request/response
✓ 9 excepciones personalizadas	Con handler centralizado y códigos HTTP apropiados
✓ Soft delete en principales entidades	Hotel, Bus, Driver, Travel
✓ 22 tests (11 service + 10 controller + 1 app)	~104 tests total
✓ Regla menor+acompañante	MinorWithoutTutorException en BookingService
✓ Regla conductor sin overlap	Query JPQL findOverlappingByDriver() en TripSegmentRepository
✓ Regla capacidad hotel/bus	Validación en BookingService.save()

10. Funcionalidades del Briefing No Implementadas

Funcionalidad	Estado	Detalle
 Email post-compra	✗	Sin spring-boot-starter-mail, sin servicio email, sin configuración SMTP

 Dashboard directivo	✗	Sin endpoints de analytics (viajes/año, ganancias, top 3)
 Frontend React	✗	Solo backend disponible
 Bus solo ida/vuelta	✗	Briefing: "desplazamiento entre hoteles no cubierto" — sin validación
 Descuento IMSERSO/colegio específico	 Parcial	Descuento grupal genérico (5% si ≥ 10 pax) no es específico IMSERSO

11. Comparativa: Auditoría Anterior (15/05) vs Estado Actual

Issues que estaban en la auditoría del 15/05 y ya están CORREGIDOS:

Issue anterior	Archivo	Estado actual
JWT filter whitelista TODAS las rutas	JwtFilter.java	✓ Solo 3 rutas públicas (/login, /api-docs, /swagger-ui)
Secret JWT hardcodeado en código	JwtUtil.java, JwtFilter.java	✓ @Value("\${jwt.secret}") desde application.properties
Tokens sin expiración	JwtUtil.java	✓ withExpiresAt() usando jwt.expiration
Login usa ID de empleado	AuthenticationController.java	✓ Login por email con @Valid
Passwords en texto plano en seed	data.sql	✓ BCrypt hashes
Tests con assertions rotas	Varios tests	✓ Mensajes de error corregidos

Issues que PERMANECEN sin corregir:

Issue	Prioridad actual
JWT secret fallback hardcodeado (default_secret_change_me)	● C1
EmployeeController sin DTO	● C2
Sin catch-all Exception	● C3
TravelService regresiones (findAll, shuffle, sin filtros)	● H1-H4
Validaciones faltantes (@Max, @NotEmpty, @Pattern)	● H7-H9
Swagger solo en CloudinaryController	● H11
README desactualizado	● H12
Naming español	● M1
Hard delete inconsistente	● M2
N+1 queries	● M5

12. Cobertura de Tests

Clase de Test	Tests	Cobertura
UserServiceTest	13	CRUD, email duplicado, tutor, delete
HotelServiceTest	14	CRUD, capacidad, soft delete
TravelServiceTest	13	CRUD, fechas, disponibles, ofertas
BookingServiceTest	16	CRUD, capacidad, menores, quote
BusServiceImplTest	9	CRUD, matrícula duplicada, soft delete
DriverServiceTest	8	CRUD, activos
EmployeeServiceTest	4	CRUD, password encryptado
OfferServiceTest	7	CRUD
TripSegmentServiceTest	8	CRUD
BookingPricingServiceTest	11	Precios, descuentos, quote
Controller Tests (10)	~15 cada uno	Endpoints HTTP con MockMvc
GIAgenciaViajesApplicationTests	1	Context load

Total: ~104 tests

Tests faltantes:

- Repository tests (@DataJpaTest)
- CloudinaryService tests
- JwtFilter / SecurityConfig tests
- Tests de integración

13. Checklist vs Briefing Original

Requisito del Briefing	Estado
4 entidades (Usuarios, hoteles, autobuses, conductor)	✅ (ampliado a 9)
CRUD para todas las entidades	✅
Figma (frontend design)	❌ No evaluable (repo backend)
Draw.io (BBDD + flujo)	❌ No evaluable
Jira para tareas	❌ No evaluable
Frontend React conectado	❌ No implementado

Frontend responsive	✗ No implementado
Manejo de excepciones	✓
DTOs	✓
Validaciones	✓ (parcial: faltan algunas)
Cloudinary	✓
Viajes en oferta (media/pensión completa)	✓
Compra múltiples plazas con nombres	✓ (vía User existente)
Tarifa niño/adulto/pensionista	✓
Viaje existente o propio	✓
Email post-compra	✗ No implementado
Vista de usuarios	✓
Dashboard directivo (año, ganancias, top 3)	✗ No implementado
Descuento IMSERSO/colegio	⚠ Parcial (group discount genérico)
Bus solo ida/vuelta	✗ No implementado
No reservar si bus/hotel completo	✓ (ambos validados)
Tests front y back	⚠ Parcial (solo back)
Aceptación: no vender pasado	✓
Aceptación: menor acompañado	✓
Aceptación: conductor 1 bus	✓

14. Prioridad de Acciones

● Inmediato (día 1)

1. **Quitar fallback hardcodeado del JWT secret** en `application.properties`
(`:default_secret_change_me`)
2. **Crear Employee DTOs** + refactor `EmployeeController` (entity → DTOs, añadir `@Valid`)
3. **Añadir handler genérico** `@ExceptionHandler(Exception.class)` en `GlobalExceptionHandler`

● Corto plazo (día 2-3)

4. **Refactorizar TravelService** para usar queries derivadas del repositorio
5. **Eliminar** `Collections.shuffle()` de los listados
6. **Arreglar** `getAll()` para filtrar solo viajes activos
7. **Arreglar** `getOnSale()` para filtrar también por fecha futura
8. **Añadir** `@Max(5)` a `stars` en `HotelRequestDTO` y `Hotel.java`
9. **Añadir** `@NotEmpty` en `customerIds` y `@NotNull` en `employeeId` de `BookingRequestDTO`
10. **Añadir** `@Transactional` a todos los métodos de `EmployeeService`

11. **Arreglar** `BookingService.update()` para reconciliar capacidad
12. **Añadir** `@Pattern` para teléfono en `DriverRequestDTO` y matrícula en `BusRequestDTO`
13. **Añadir** `@Repository` a los 5 repositorios que faltan

● Medio plazo (sprint)

14. **Añadir Swagger** (`@Operation` , `@ApiResponse`) a todos los controllers
15. **Crear BusMapper** y eliminar lógica manual de `BusServiceImpl`
16. **Arreglar N+1 queries** (usar `findAllById()` en vez de loop `findById()`)
17. **Arreglar tipos** "l viaje" → "el viaje" en todas las excepciones
18. **Limpiar UserResponseDTO** (redundancias, userId/id duplicados)
19. **Renombrar métodos en español** a inglés (`algoritmo` → `algorithm` , `reducirPlazas` → `reduceCapacity` , etc.)
20. **Arreglar posible mismatch de ruta Swagger** en `JwtFilter`
21. **Unificar hard delete** → **soft delete** en `UserService`
22. **Implementar envío de email** con `spring-boot-starter-mail`
23. **Crear endpoints de Dashboard**: viajes/año, ganancias/año, top 3 viajes
24. **Actualizar README.md** para reflejar el stack real

● Largo plazo

25. **Añadir paginación** (`Pageable`) a endpoints GET
26. **Crear tests de integración** con `@SpringBootTest` + H2
27. **Crear tests de repository** con `@DataJpaTest`
28. **Añadir perfil de test** con H2 en memoria
29. **Versionado de API** (`/api/v1/`)

15. Conclusión

El proyecto tiene una **base sólida**: arquitectura limpia, motor de precios robusto, buena cobertura de tests unitarios, manejo de excepciones bien estructurado, y los problemas críticos de seguridad de la auditoría anterior han sido corregidos.

Sin embargo, persisten **3 issues críticos** (JWT fallback hardcodeado, `EmployeeController` sin DTO, sin catch-all Exception handler) y **12 issues de alta prioridad** que afectan a la calidad del código, rendimiento y mantenibilidad.

Además, **funcionalidades clave del briefing** (email post-compra, dashboard directivo, frontend React) **no están implementadas**.

Prioridades:

1. ● Corregir los 3 issues críticos de seguridad y calidad
2. ● Refactorizar `TravelService` y añadir validaciones faltantes
3. ● Añadir Swagger, limpiar código, unificar naming
4. ● Implementar funcionalidades faltantes del briefing

Puntuación general: 7/10 — Bien estructurado pero con gaps de seguridad menores y funcionalidades pendientes.